TÀI LIỆU KỸ THUẬT SỐ 01 / BỘ 04 TÀI LIỆU TRIỂN KHAI

KẾ HOẠCH XỬ LÝ VÀ CHUẨN BỊ DỮ LIỆU

Quy trình làm sạch, đóng gói và chuẩn hóa dữ liệu huấn luyện cho 3 tác tử thay thế hệ thống MARS-SQL

> **Mục tiêu cốt lõi của quy trình dữ liệu:** Khi giảm quy mô hệ thống từ 21 tỷ tham số (3 mô hình 7B) xuống còn 2.5 tỷ – 4.0 tỷ tham số, các mô hình ngôn ngữ nhỏ (SLM) rất nhạy cảm với dữ liệu bẩn và câu lệnh giải thích dài dòng. Quy trình dưới đây biến các tập dữ liệu thô thành 4 gói dữ liệu sạch chuyên biệt, giúp mô hình nhỏ học đúng trọng tâm ngay từ bản Demo đầu tiên.

1. Tổng Quan Các Nguồn Dữ Liệu Đầu Vào

Để đảm bảo tính công bằng khi so sánh với hệ thống gốc MARS-SQL **Yang et al. (2026)** và tận dụng tối đa sức mạnh của các mô hình nhỏ trong SLM-SQL **Sheng & Xu (2025a)**, dự án sử dụng 4 nguồn dữ liệu chuẩn gồm: bộ dữ liệu thực tế quy mô lớn **BIRD**, bộ dữ liệu đa miền **Spider 1.0**, bộ kiểm thử suy luận ngoài miền **Spider-DK** và kho dữ liệu suy luận tổng hợp **SynSQL-2.5M**.

Tên Bộ Dữ Liệu	Quy Mô & Dung Lượng	Đặc Điểm Thực Tế	Vai Trò Trong Dự Án
BIRD (v1.0) Li et al. (2023)	12,751 cặp câu hỏi–SQL trên 95 cơ sở dữ liệu (nặng 33.4 GB)	Trải rộng trên 37 lĩnh vực (tài chính, y tế, giáo dục...), dữ liệu chứa nhiều nhiễu và cần kết hợp gợi ý kiến thức bên ngoài (external knowledge evidence).	Nguồn dữ liệu chính để huấn luyện (sau khi lọc sạch còn 8,036 mẫu) và đo kiểm hiệu năng thực tế trên tập `dev_20240627`.
Spider 1.0 Yu et al. (2018)	Tập Dev và tập Test gồm 2,147 mẫu trên 138 lĩnh vực	Lược đồ cơ sở dữ liệu được thiết kế sạch sẽ, tên bảng và cột rõ ràng, không trùng lặp cơ sở dữ liệu giữa tập học và tập thi.	Đánh giá khả năng gặp cơ sở dữ liệu mới hoàn toàn (zero-shot cross-domain) mà mô hình chưa từng nhìn thấy lúc học.
Spider-DK (v1.0) Gan et al. (2021)	Biến thể mở rộng từ tập Spider chuẩn	Bổ sung các câu hỏi đòi hỏi hiểu biết ngầm định về kiến thức chuyên ngành mà không ghi sẵn trong đề bài.	Kiểm tra độ bền vững khi chuyển giao sang miền dữ liệu khác (Out-of-Domain / OOD transfer) mà không được học lại.
SynSQL-2.5M Sheng & Xu (2025a)	2.5 triệu mẫu tổng hợp kèm lời giải từng bước (CoT)	Do phương pháp OmniSQL sinh tự động nên còn lẫn nhiều câu giải thích thừa, lặp lại câu lệnh hoặc thiếu chuẩn mực.	Lọc sạch để lấy 916,000 mẫu dạy viết code SQL và 310,000 mẫu dạy gộp sửa lỗi cho các mô hình nhỏ 0.5B–3B.

2. Bước 1: Lọc Bỏ Nhãn Sai Trên Tập Huấn Luyện BIRD (9,428 → 8,036 Mẫu)

Trong tập huấn luyện gốc của BIRD (gồm 9,428 mẫu), có một tỷ lệ đáng kể các câu lệnh SQL đáp án (gold SQL) bị người gán nhãn viết sai hoặc không khớp với câu hỏi. Nếu dùng nguyên tập dữ liệu này để dạy mô hình nhỏ hoặc chấm điểm thưởng trong Học tăng cường (RL), mô hình sẽ bị phạt oan khi viết đúng và được thưởng khi bắt chước lỗi sai của đáp án.

Cách thực hiện cụ thể: Áp dụng đúng quy trình kiểm tra chéo của MARS-SQL **Yang et al. (2026)** bằng cách dùng hai mô hình lớn độc lập là **Gemini-2.5-pro** và **GPT-4o** rà soát lại toàn bộ 9,428 mẫu huấn luyện của BIRD. Bất kỳ mẫu nào có đáp án gốc bị cả hai hệ thống phát hiện mâu thuẫn ngữ nghĩa với câu hỏi hoặc trả về kết quả rỗng bất thường đều bị loại bỏ. Kết quả thu được bộ khung **8,036 mẫu huấn luyện sạch chuẩn hóa** làm gốc cho cả 3 tác tử.

3. Bước 2: Chuẩn Bị Dữ Liệu Cho Agent 1 (Tìm Bảng, Cột & Đường Nối)

Ở Giai đoạn 1 (Schema Grounding), nhiệm vụ của hệ thống là nhìn vào câu hỏi của người dùng và nhặt ra đúng các bảng, các cột cần thiết từ hàng chục bảng trong cơ sở dữ liệu. Thay vì bắt mô hình đọc đi đọc lại toàn bộ bảng mỗi lần có câu hỏi mới, chúng ta chuẩn bị hai gói dữ liệu cho phương pháp **LitE-SQL** **Piao et al. (2026)** và **Graph-Link** **Zhong et al. (2026)**:

3.1. Đóng gói 90,102 thẻ thông tin cột và tạo "Mẫu âm khó" (Hard Negatives)

Từ 8,036 mẫu huấn luyện sạch của BIRD, hệ thống tách từng bảng ra độc lập và thu được tổng cộng **90,102 điểm dữ liệu cấp độ bảng và cột**. Để mô hình nhúng siêu nhẹ `Qwen3-Embedding-0.6B` hiểu rõ từng cột dùng để làm gì, mỗi cột được gom thành một đoạn văn bản mô tả đầy đủ gồm 6 thông tin:

1. **Tên bảng chứa cột** (ví dụ: bảng `schools` hoặc bảng `frpm`).
2. **Tên cột chính xác** (ví dụ: `CDSCode`, `County Name`).
3. **Kiểu dữ liệu** (ví dụ: `TEXT`, `INTEGER`, `REAL`, `DATE`).
4. **Ràng buộc khóa** (ghi rõ cột có phải là Khóa chính `PRIMARY KEY` hay Khóa ngoại `FOREIGN KEY` nối sang bảng nào không).
5. **Mô tả ý nghĩa cột** (lấy từ file từ điển dữ liệu đi kèm cơ sở dữ liệu).
6. **Các giá trị mẫu đại diện** (lấy 3–5 giá trị thực tế đang lưu trong cột đó để mô hình biết định dạng dữ liệu).

Cách gán nhãn dạy phân biệt cột trùng tên (HN-SupCon): Trong một cơ sở dữ liệu thực tế, rất nhiều bảng cùng có các cột mang tên giống hệt nhau như `id`, `name`, `status`, `date`. Khi đối chiếu với câu lệnh SQL đáp án chuẩn:

- **Nhãn Dương (`y = 1`):** Gán cho tất cả các cột thực sự xuất hiện trong câu lệnh SQL đáp án.
- **Mẫu Âm Khó (Hard Negatives, `y = 0`):** Quét toàn bộ các cột còn lại trong cùng cơ sở dữ liệu đó. Những cột nào không nằm trong đáp án nhưng có độ tương đồng ngữ nghĩa cao với câu hỏi (hoặc trùng tên với cột đúng) được gom riêng thành tập "mẫu âm khó" với biên phân cách `margin = 0.1` . Khi học trên tập này, mô hình 0.6B sẽ bị ép phải nhìn kỹ tên bảng và ý nghĩa cột chứ không chọn bừa theo tên cột.

3.2. Dựng bản đồ đường nối Khóa chính – Khóa ngoại (PK-FK Graph)

Để khắc phục lỗi mất "bảng cầu nối" ở 48 câu hỏi trên tập BIRD-dev của MARS-SQL gốc, chúng ta viết một kịch bản Python quét toàn bộ các câu lệnh tạo bảng (`CREATE TABLE` DDL) của từng cơ sở dữ liệu để dựng sẵn đồ thị tĩnh **G = (V, E)**:

- **Tập nút (V):** Gồm toàn bộ các nút Bảng và các nút Cột trong cơ sở dữ liệu.
- **Tập cạnh nối (E):** Gồm 2 loại đường nối: đường nối nội bộ từ một Bảng đến các Cột thuộc bảng đó, và đường nối liên kết Khóa chính – Khóa ngoại (PK-FK) giữa hai cột của hai bảng khác nhau. Bản đồ này được lưu thành file cấu trúc nhẹ trên ổ cứng để thuật toán **Cây Steiner (Graph-Link)** tra cứu tức thì khi chạy thực tế.

4. Bước 3: Làm Sạch Dữ Liệu Suy Luận Cho Agent 2 (Viết Câu Lệnh SQL)

Các mô hình ngôn ngữ nhỏ (từ 0.5B đến 3B như `SLM-SQL-1.5B` hay `FINER-SQL-3B`) có bộ nhớ hạn chế. Nếu đưa cho chúng dữ liệu huấn luyện có phần suy luận lan man hoặc câu lệnh SQL chứa bình luận rác, mô hình nhỏ sẽ nhanh chóng bắt chước thói quen viết dài dòng và sinh lỗi cú pháp.

4.1. Bốn bộ lọc làm sạch kho dữ liệu SynSQL-2.5M thành SynSQL-Think-916K

Áp dụng đúng quy trình của SLM-SQL **Sheng & Xu (2025a)** trên 2.5 triệu mẫu của tập SynSQL-2.5M, chúng ta chạy kịch bản lọc tự động loại bỏ ngay lập tức bất kỳ mẫu nào vi phạm 1 trong 4 điều kiện sau:

1. **Thiếu từ khóa truy vấn:** Câu lệnh SQL đầu ra không chứa từ khóa `SELECT` hợp lệ.
2. **Lặp lại câu lệnh trong lúc nghĩ:** Phần suy luận từng bước (Chain-of-Thought) viết lặp đi lặp lại nguyên cả câu lệnh SQL nhiều lần gây tốn token vô ích.
3. **Chứa chú thích trong code:** Câu lệnh SQL bên trong có chứa ký hiệu chú thích `--` (rất dễ làm đứt gãy bộ kiểm tra cú pháp hoặc gây lỗi khi ghép chuỗi).
4. **Đề bài quá dài:** Tổng chiều dài của phần đề bài (prompt gồm câu hỏi và mô tả bảng) vượt quá **7,000 token**.

Sau khi lọc xong, kịch bản tiếp tục cắt bỏ toàn bộ phần văn bản giải thích dư thừa nằm phía sau câu lệnh SQL. Toàn bộ phần suy luận phía trước được bọc gọn vào cặp thẻ `<think>...</think>` và câu lệnh SQL chuẩn được đặt vào cặp thẻ `<answer>...</answer>` (hoặc `<SQL>...</SQL>`). Quá trình này tạo ra hai bộ dữ liệu tinh gọn:

- **SynSQL-Think-916K (916,000 mẫu):** Dùng để dạy mô hình chính cách suy luận từng bước và viết câu lệnh SQL chuẩn.
- **SynSQL-Merge-Think-310K (310,000 mẫu):** Mỗi mẫu gồm nhiều câu lệnh SQL ứng viên có kết quả khác nhau kèm phân tích điểm đúng/sai và câu lệnh SQL hợp nhất cuối cùng, dùng để dạy mô hình gộp sửa lỗi (Merge Revision Model 0.5B) trong phương pháp CSC-SQL **Sheng & Xu (2025b)**.

4.2. Xây dựng Ngân hàng lời giải mẫu (Reasoning Bank) và cặp góc nhìn LLM-JEPA

Đối với mô hình `FINER-SQL-3B` **Hoang et al. (2026)**, trên 8,036 mẫu BIRD sạch, chúng ta thu thập các luồng suy luận mẫu chất lượng cao từ các mô hình suy luận lớn (như DeepSeek-R1), chỉ giữ lại những lời giải mà câu lệnh SQL chạy ra kết quả khớp 100% với đáp án chuẩn để lập thành **Reasoning Bank**. Ngân hàng này vừa dùng để dạy bước đầu (SFT), vừa được nhúng thành vector để chấm điểm thưởng "suy nghĩ hợp lý" (Memory Reward) trong lúc học tăng cường.

Đồng thời, để phục vụ kỹ thuật căn chỉnh não bộ **LLM-JEPA** **Huang et al. (2026)** ở pha nâng cao, mỗi mẫu trong 8,036 câu BIRD được đóng gói thành 2 khối nằm chung trong một cửa sổ văn bản: **Khối 1 (Text)** gồm câu hỏi `Q`, gợi ý `E`, lược đồ rút gọn `S'` nối thêm `k = 1` token đặc biệt `[PRED]` ở cuối; **Khối 2 (Code)** chứa câu lệnh SQL đáp án chuẩn.

5. Bước 4: Sinh Quỹ Đạo Ứng Viên & Bộ Lọc An Toàn Cho Agent 3

Ở Giai đoạn 3 (Validation), các mô hình quyết định nhanh System-1 (như `NanoJev-0.6B`, `Laya-421M`, `Kev-0.8B`) có nhiệm vụ nhìn vào 8 phương án SQL do Agent 2 viết ra và chấm điểm chọn phương án đúng nhất chỉ trong vài mili-giây.

5.1. Sinh 16 quỹ đạo mẫu (Rollouts) trên mỗi câu hỏi để tạo tập huấn luyện chấm điểm

Trên 8,036 câu hỏi của tập huấn luyện BIRD, chúng ta cho Agent 2 chạy sinh thử **16 quỹ đạo tương tác (trajectories) cho mỗi câu hỏi** với bộ thông số lấy mẫu đa dạng: nhiệt độ `Temperature T = 0.7`, `Top-p = 0.9` và `Top-k = 50`. Toàn bộ 16 câu lệnh SQL sinh ra của mỗi câu hỏi được đem chạy thật trên cơ sở dữ liệu và so sánh bảng kết quả với đáp án chuẩn:

- Phương án nào chạy không lỗi và trả về bảng dữ liệu trùng khớp 100% với đáp án chuẩn được gán nhãn **Đúng (`1`)**.
- Phương án nào báo lỗi cú pháp, lỗi tên bảng hoặc trả về kết quả sai lệch được gán nhãn **Sai (`0`)**.
- Những câu hỏi mà cả 16 phương án sinh ra đều sai (toàn nhãn `0`) được gom riêng để dạy cho mô hình `Kev-0.8B` kỹ năng **"Từ chối phán quyết khi mọi phương án đều đáng ngờ" (Abstention)**.

5.2. Trích xuất từ khóa cho 10 quy tắc kiểm tra tĩnh PV-SQL và Lọc bảo mật

Theo phương pháp **PV-SQL** **Tian & Zhang (2026)**, chúng ta thiết lập bộ từ điển ánh xạ các từ khóa trong câu hỏi tiếng tự nhiên sang 10 quy tắc kiểm tra cây cú pháp (AST) tất định (ví dụ: câu hỏi có chữ `unique`, `distinct`, `different` thì bắt buộc cây cú pháp SQL phải có `DISTINCT` hoặc `GROUP BY`; có chữ `top N`, `highest` thì phải có `ORDER BY` kèm `LIMIT`; có chữ `average` thì phải có hàm `AVG()`).

Quy tắc lọc bảo mật bắt buộc (Chống tấn công Prompt Injection trên System-1): Nghiên cứu bảo mật của **Wang et al. (2026)** chỉ ra rằng các mô hình dòng Jev / NanoJev rất dễ bị đánh lừa (tỷ lệ bị tấn công thành công lên tới 96.8%–100%) nếu kẻ xấu chèn một đoạn chữ độc hại vào trong các ô dữ liệu văn bản của bảng cơ sở dữ liệu, rồi đoạn chữ đó vô tình được in ra trong kết quả chạy thử SQL. Vì vậy, kịch bản chuẩn bị dữ liệu cho Agent 3 bắt buộc phải **tách biệt nghiêm ngặt**: chỉ đưa tên bảng/cột tin cậy, câu lệnh SQL và biên bản kiểm tra 10 quy tắc AST vào phần trạng thái `state` của mô hình System-1; tuyệt đối không nối trực tiếp các chuỗi văn bản thô dài lấy từ ô dữ liệu người dùng vào câu lệnh điều khiển của bộ chấm điểm.

6. Bảng Tổng Hợp Các Gói Dữ Liệu Đầu Ra Để Bàn Giao Cho Huấn Luyện

Mã Gói Dữ Liệu	Phục Vụ Tác Tử	Số Lượng Mẫu Chuẩn	Định Dạng Lưu Trữ & Mục Đích Sử Dụng
DATA-G1-EMB	Agent 1 (LitE-SQL 0.6B)	90,102 điểm dữ liệu cột (từ 8,036 câu BIRD)	Cặp `(Câu hỏi Q, Thẻ thông tin cột)` kèm nhãn dương và tập mẫu âm khó (`margin = 0.1`) để train bộ nhúng cột.
DATA-G1-GRAPH	Agent 1 (Graph-Link Steiner)	95 đồ thị DB BIRD + 138 đồ thị DB Spider	Danh sách nút (Bảng, Cột) và cạnh nối (PK-FK) trích xuất từ DDL để giải bài toán Cây Steiner giữ bảng cầu nối.
DATA-G2-SFT	Agent 2 (SLM-SQL / FINER-SQL / CodeS)	916,000 mẫu SynSQL + 8,036 mẫu Reasoning Bank	Định dạng chuẩn `<think>...</think><answer>...</answer>` đã lọc sạch 4 lỗi và gắn cờ chỉ tính loss trên phần trả lời.
DATA-G2-MERGE	Agent 2 (Bộ gộp CSC-SQL 0.5B)	310,000 mẫu xung đột (SynSQL-Merge-Think)	Gồm danh sách các câu lệnh SQL bất đồng kết quả và câu lệnh hợp nhất đúng để dạy mô hình sửa lỗi tập thể.
DATA-G3-VAL	Agent 3 (NanoJev / Laya / Kev)	8,036 × 16 = 128,576 quỹ đạo ứng viên SQL	Đã gắn nhãn thực thi thật `{0, 1}` , kèm biên bản 10 quy tắc AST và đã lọc tách biệt dữ liệu thô chống chèn lệnh độc hại.

Tài Liệu Tham Khảo

1. Yang, H., Zhang, J., He, Z., Zhou, A., & Fung, Y. R. (2026). *MARS-SQL: Multi-Agent Reinforcement Learning for Text-to-SQL*. ICML 2026 (arXiv:2511.01008).
2. Li, J., et al. (2023). *Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)*. NeurIPS 2023 (arXiv:2305.03111).
3. Yu, T., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. EMNLP 2018.
4. Gan, Y., et al. (2021). *Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization (Spider-DK)*. ACL 2021.
5. Piao, S., Lee, J., & Park, S. (2026). *LitE-SQL: A Lightweight and Efficient Text-to-SQL Framework with Vector-based Schema Linking and Execution-Guided Self-Correction*. Findings of EACL 2026 (arXiv:2510.09014).
6. Zhong, J., Yang, Y., Liu, Q., Xu, R., Chi, M., & Mo, Y. (2026). *Graph-Link: Bridging the Semantic-Structural Gap in Text-to-SQL via Constrained Subgraph Induction*. ICML 2026.

7. Sheng, L., & Xu, S.-S. (2025a). *SLM-SQL: An Exploration of Small Language Models for Text-to-SQL*. Preprint (arXiv:2507.22478).
8. Sheng, L., & Xu, S.-S. (2025b). *CSC-SQL: Corrective Self-Consistency in Text-to-SQL via Reinforcement Learning*. Preprint (arXiv:2505.13271).
9. Hoang, T. D., et al. (2026). *FINER-SQL: Boosting Small Language Models for Text-to-SQL with Fine-Grained Execution Feedback and Cost-Efficient Rewards*. IEEE ICDE 2026 (arXiv:2605.03465).
10. Huang, H., LeCun, Y., & Balestriero, R. (2026). *LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures*. ICLR 2026 (arXiv:2509.14252).
11. Tian, Y., & Zhang, T. (2026). *PV-SQL: Synergizing Database Probing and Rule-based Verification for Text-to-SQL Agents*. Findings of ACL 2026 (arXiv:2604.17653).
12. Wang, S., Zhu, T., Chen, H., Li, J., Yang, M., & Liu, B. (2026). *Hidden Risks of Jev: An Empirical Study of Security, Privacy, and Dual Use*. Preprint (arXiv:2610.04985).